

Add description for parallel `<memory>` algorithms

Document #: P3890R0 [[Latest](#)] [[Status](#)]
Date: 2025-11-07
Project: Programming Language C++
Audience: LWG
Reply-to: Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>

Contents

- [1 Motivation](#)
- [2 Formal wording](#)
 - [2.1 Modify \[\[specialized.algorithms.general\]\(#\)\]](#)
 - [2.2 Modify \[\[uninitialized.construct.default\]\(#\)\]](#)
 - [2.3 Modify \[\[uninitialized.construct.value\]\(#\)\]](#)
 - [2.4 Modify \[\[uninitialized.copy\]\(#\)\]](#)
 - [2.5 Modify \[\[uninitialized.move\]\(#\)\]](#)
 - [2.6 Modify \[\[uninitialized.fill\]\(#\)\]](#)
 - [2.7 Modify \[\[specialized.destroy\]\(#\)\]](#)
- [3 References](#)

§ Abstract

This paper adds the algorithmic description for parallel `<memory>` algorithms.

§ 1 Motivation

C++ standard has parallel `<memory>` algorithms since C++17. [[P3179R9](#)] adds overloads for the parallel `<memory>` algorithms for C++26. However, the problem is that those algorithms exist in `<memory>` header synopsis only ([[memory.syn](#)]) and were never described in [[specialized.algorithms](#)], which is a dedicated place for algorithmic description of `<memory>` algorithms (not only parallel).

This proposal aims to fix that.

§ 2 Formal wording

§ 2.1 Modify [\[specialized.algorithms.general\]](#)

- 3 [*Note*: When new objects are created by the algorithms specified in [\[specialized.algorithms\]](#), the lifetime ends for any existing objects (including potentially-overlapping sub-objects [\[intro.object\]](#)) in storage that is reused [\[basic.life\]](#). — *end note*]
- 4 Some algorithms specified in [\[specialized.algorithms\]](#) make use of the following exposition-only function templates:

```
template<class T>
constexpr void* voidify(T& obj) noexcept {
    return addressof(obj);
}

template<class I>
decltype(auto) deref-move(I& it) {
    if constexpr (is_lvalue_reference_v<decltype(*it)>)
        return std::move(*it);
    else
        return *it;
}
```

- 5 For the parallel algorithm overloads in subclause [\[specialized.algorithms\]](#), each iteration of the loop with a new expression or invocation of `destroy`” in the specification of the function template (if any) is considered to be an invocation of an element access function ([\[algorithms.parallel.exec\]](#)).

§ 2.2 Modify [\[uninitialized.construct.default\]](#)

```
template<class NoThrowForwardIterator>
constexpr void uninitialized_default_construct(NoThrowForwardIterator
    first,
                                           NoThrowForwardIterator
    last);
+ template<class ExecutionPolicy, class NoThrowForwardIterator>
+ void uninitialized_default_construct(ExecutionPolicy&& exec,
+                                     NoThrowForwardIterator first,
+                                     NoThrowForwardIterator last);
```

- 1 *Effects*: Equivalent to:

```
for (; first != last; ++first)
    ::new (voidify(*first))
        typename iterator_traits<NoThrowForwardIterator>::value_type;
```

```
namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_default_construct(I first, S last);
    template<nothrow-forward-range R>
```

```

    requires default_initializable<range_value_t<R>>
    constexpr borrowed_iterator_t<R> uninitialized_default_construct(R&&
        r);

+ template<execution-policy Ep, nothrow-random-access-iterator I,
+         nothrow-sized-sentinel-for<I> S>
+     requires default_initializable<iter_value_t<I>>
+     I uninitialized_default_construct(Ep&& exec,
+         I first, S last);
+ template<execution-policy Ep, nothrow-sized-random-access-range R>
+     requires default_initializable<range_value_t<R>>
+     borrowed_iterator_t<R> uninitialized_default_construct(Ep&& exec,
+         R&& r);
}

```

2 *Effects:* Equivalent to:

```

for (; first != last; ++first)
    ::new (voidify(*first)) remove_reference_t<iter_reference_t<I>>;
return first;

```

```

template<class NoThrowForwardIterator, class Size>
constexpr NoThrowForwardIterator
    uninitialized_default_construct_n(NoThrowForwardIterator first, Size
        n);
+ template<class ExecutionPolicy, class NoThrowForwardIterator, class
+         Size>
+     NoThrowForwardIterator
+     uninitialized_default_construct_n(ExecutionPolicy&& exec,
+         NoThrowForwardIterator first,
+         Size n);

```

3 *Effects:* Equivalent to:

```

for (; n > 0; (void)++first, --n)
    ::new (voidify(*first))
        typename iterator_traits<NoThrowForwardIterator>::value_type;
return first;

```

```

namespace ranges {
    template<nothrow-forward-iterator I>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_default_construct_n(I first, iter_differ-
            ence_t<I> n);
}

```

4 *Effects:* Equivalent to:

```

return uninitialized_default_construct(counted_iterator(first, n),
    default_sentinel).base();

```

```

namespace ranges {
    template<execution-policy Ep, nothrow-random-access-iterator I>
        requires default_initializable<iter_value_t<I>>
        I uninitialized_default_construct_n(Ep&& exec, I first,

```

```

iter_difference_t<I> n);
}

```

5 *Effects:* Equivalent to:

```

return uninitialized_default_construct(std::forward<Ep>(exec), count-
    ed_iterator(first, n),
    default_sentinel).base();

```

§ 2.3 Modify [uninitialized.construct.value]

```

template<class NoThrowForwardIterator>
constexpr void uninitialized_value_construct(NoThrowForwardIterator
    first,
    NoThrowForwardIterator
    last);
+ template<class ExecutionPolicy, class NoThrowForwardIterator>
+ void uninitialized_value_construct(ExecutionPolicy&& exec,
+     NoThrowForwardIterator first,
+     NoThrowForwardIterator last)

```

1 *Effects:* Equivalent to:

```

for (; first != last; ++first)
    ::new (voidify(*first))
        typename iterator_traits<NoThrowForwardIterator>::value_type();

```

```

namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_value_construct(I first, S last);
    template<nothrow-forward-range R>
        requires default_initializable<range_value_t<R>>
        constexpr borrowed_iterator_t<R> uninitialized_value_construct(R&& r);

+ template<execution-policy Ep, nothrow-random-access-iterator I,
+     nothrow-sized-sentinel-for<I> S>
+     requires default_initializable<iter_value_t<I>>
+     I uninitialized_value_construct(Ep&& exec,
+         I first, S last);
+ template<execution-policy Ep, nothrow-sized-random-access-range R>
+     requires default_initializable<range_value_t<R>>
+     borrowed_iterator_t<R> uninitialized_value_construct(Ep&& exec,
+         R&& r);
}

```

2 *Effects:* Equivalent to:

```

for (; first != last; ++first)
    ::new (voidify(*first)) remove_reference_t<iter_reference_t<I>>();
return first;

```

```

template<class NoThrowForwardIterator, class Size>
constexpr NoThrowForwardIterator
    uninitialized_value_construct_n(NoThrowForwardIterator first, Size n);

```

```
+ template<class ExecutionPolicy, class NoThrowForwardIterator, class
    Size>
+ NoThrowForwardIterator
+ uninitialized_value_construct_n(ExecutionPolicy&& exec,
+                               NoThrowForwardIterator first,
+                               Size n);
```

3 *Effects:* Equivalent to:

```
for (; n > 0; (void)++first, --n)
    ::new (voidify(*first))
        typename iterator_traits<NoThrowForwardIterator>::value_type();
return first;

namespace ranges {
    template<nothrow-forward-iterator I>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_value_construct_n(I first, iter_difference_t<I> n);
}
```

4 *Effects:* Equivalent to:

```
return uninitialized_value_construct(counted_iterator(first, n),
                                    default_sentinel).base();

namespace ranges {
    template<execution-policy Ep, nothrow-random-access-iterator I>
        requires default_initializable<iter_value_t<I>>
        I uninitialized_value_construct_n(Ep&& exec, I first,
                                          iter_difference_t<I> n);
}
```

5 *Effects:* Equivalent to:

```
return uninitialized_value_construct(std::forward<Ep>(exec), counted_iterator(first, n),
                                    default_sentinel).base();
```

§ 2.4 Modify `[uninitialized.copy]`

```
template<class InputIterator, class NoThrowForwardIterator>
    constexpr NoThrowForwardIterator uninitialized_copy(InputIterator first,
        InputIterator last,
        NoThrowForwardIterator result);
+ template<class ExecutionPolicy, class ForwardIterator, class
    NoThrowForwardIterator>
+ NoThrowForwardIterator uninitialized_copy(ExecutionPolicy&& exec,
+                                         ForwardIterator first,
+                                         ForwardIterator last,
+                                         NoThrowForwardIterator
        result);
```

1 *Preconditions:* `result + [0, (last - first))` does not overlap with `[first, last)`.

- 2 *Effects*: Equivalent to:

```
for (; first != last; ++result, (void)++first)
    ::new (voidify(*result))
        typename iterator_traits<NoThrowForwardIterator>::value_
            type(*first);
```

- 3 *Returns*: result.

```
namespace ranges {
    template<input_iterator I, sentinel_for<I> S1,
            nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
        requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
        constexpr uninitialized_copy_result<I, O>
            uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast);
    template<input_range IR, nothrow-forward-range OR>
        requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
        constexpr uninitialized_copy_result<borrowed_iterator_t<IR>,
            borrowed_iterator_t<OR>>
            uninitialized_copy(IR&& in_range, OR&& out_range);

+   template<execution-policy Ep, random_access_iterator I, sized_sen-
+       tinel_for<I> S1,
+       nothrow-random-access-iterator O, nothrow-sized-sentinel-
+       for<O> S2>
+       requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
+       uninitialized_copy_result<I, O>
+       uninitialized_copy(Ep&& exec, I ifirst, S1 ilast,
+       O ofirst, S2 olast);
+   template<execution-policy Ep, sized-random-access-range IR,
+       nothrow-sized-random-access-range OR>
+       requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
+       uninitialized_copy_result<borrowed_iterator_t<IR>,
+       borrowed_iterator_t<OR>>
+       uninitialized_copy(Ep&& exec, IR&& in_range,
+       OR&& out_range);
}
```

- 4 *Preconditions*: [ofirst, olast) does not overlap with [ifirst, ilast).

- 5 *Effects*: Equivalent to:

```
for (; ifirst != ilast && ofirst != olast; ++ofirst, (void)++ifirst)
    ::new (voidify(*ofirst)) remove_reference_t<iter_reference_t<O>>
        (*ifirst);
return {std::move(ifirst), ofirst};
```

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
    constexpr NoThrowForwardIterator uninitialized_copy_n(InputIterator
        first, Size n,

        NoThrowForwardIterator result);
+ template<class ExecutionPolicy, class ForwardIterator, class Size,
+     class NoThrowForwardIterator>
+     NoThrowForwardIterator uninitialized_copy_n(ExecutionPolicy&& exec,
+     ForwardIterator first,
```

```

+                                     Size n,
+                                     NoThrowForwardIterator
    result);

```

6 *Preconditions:* `result + [0, n)` does not overlap with `first + [0, n)`.

7 *Effects:* Equivalent to:

```

for (; n > 0; ++result, (void)++first, --n)
    ::new (voidify(*result))
        typename iterator_traits<NoThrowForwardIterator>::value_
            type(*first);

```

8 *Returns:* `result`.

```

namespace ranges {
    template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-
        for<O> S>
        requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
        constexpr uninitialized_copy_n_result<I, O>
            uninitialized_copy_n(I ifirst, iter_difference_t<I> n, O ofirst, S
                olast);
}

```

9 *Preconditions:* `[ofirst, olast)` does not overlap with `ifirst + [0, n)`.

10 *Effects:* Equivalent to:

```

auto t = uninitialized_copy(counted_iterator(std::move(ifirst), n),
                            default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};

```

```

template<execution-policy Ep, random_access_iterator I, nothrow-random-ac-
    cess-iterator O,
        nothrow-sized-sentinel-for<O> S>
    requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
    uninitialized_copy_n_result<I, O>
        uninitialized_copy_n(Ep&& exec, I ifirst, iter_difference_t<I> n,
            O ofirst, S olast);

```

x *Preconditions:* `[ofirst, olast)` does not overlap with `ifirst + [0, n)`.

x *Effects:* Equivalent to:

```

auto t = uninitialized_copy(std::forward<Ep>(exec),
                            counted_iterator(std::move(ifirst), n),
                            default_sentinel, ofirst, olast);
return {t.in.base(), t.out};

```

§ 2.5 Modify `[uninitialized.move]`

```

template<class InputIterator, class NoThrowForwardIterator>
    constexpr NoThrowForwardIterator uninitialized_move(InputIterator first,
        InputIterator last,

```

```

        NoThrowForwardIterator result);
+   template<class ExecutionPolicy, class ForwardIterator, class
+       NoThrowForwardIterator>
+   NoThrowForwardIterator uninitialized_move(ExecutionPolicy&& exec,
+       ForwardIterator first,
+       ForwardIterator last,
+       NoThrowForwardIterator
+       result);

```

1 *Preconditions:* result + [0, (last - first)) does not overlap with [first, last).

2 *Effects:* Equivalent to:

```

for (; first != last; (void)++result, ++first)
    ::new (voidify(*result))
        typename iterator_traits<NoThrowForwardIterator>::value_type(deref-
            move(first));
return result;

```

```

namespace ranges {
    template<input_iterator I, sentinel_for<I> S1,
        nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
        requires constructible_from<iter_value_t<O>,
            iter_rvalue_reference_t<I>>
        constexpr uninitialized_move_result<I, O>
            uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);
    template<input_range IR, nothrow-forward-range OR>
        requires constructible_from<range_value_t<OR>,
            range_rvalue_reference_t<IR>>
        constexpr uninitialized_move_result<borrowed_iterator_t<IR>,
            borrowed_iterator_t<OR>>
            uninitialized_move(IR&& in_range, OR&& out_range);

+   template<execution-policy Ep, random_access_iterator I, sized_sen-
+       tinel_for<I> S1,
+       nothrow-random-access-iterator O, nothrow-sized-sentinel-for<O>
+       S2>
+       requires constructible_from<iter_value_t<O>,
+       iter_rvalue_reference_t<I>>
+       uninitialized_move_result<I, O>
+       uninitialized_move(Ep&& exec, I ifirst, S1 ilast,
+       O ofirst, S2 olast);
+   template<execution-policy Ep, sized-random-access-range IR, nothrow-
+       sized-random-access-range OR>
+       requires constructible_from<range_value_t<OR>,
+       range_rvalue_reference_t<IR>>
+       uninitialized_move_result<borrowed_iterator_t<IR>,
+       borrowed_iterator_t<OR>>
+       uninitialized_move(Ep&& exec, IR&& in_range,
+       OR&& out_range);
+
+   }
}

```

3 *Preconditions:* [ofirst, olast) does not overlap with [ifirst, ilast).

4 *Effects:* Equivalent to:


```

for (; ifirst != ilast && ofirst != olast; ++ofirst, (void)++ifirst)
    ::new (voidify(*ofirst))
        remove_reference_t<iter_reference_t<0>>>(ranges::iter_move(ifirst));
return {std::move(ifirst), ofirst};

```

- 5 [*Note:* If an exception is thrown, some objects in the range [ifirst, ilast) are left in a valid, but unspecified state. — *end note*]

```

template<class InputIterator, class Size, class NoThrowForwardIterator>
constexpr pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n,
        NoThrowForwardIterator result);
+ template<class ExecutionPolicy, class ForwardIterator, class Size,
+     class NoThrowForwardIterator>
+     pair<ForwardIterator, NoThrowForwardIterator>
+     uninitialized_move_n(ExecutionPolicy&& exec,
+         ForwardIterator first, Size n,
+         NoThrowForwardIterator result);

```

- 6 *Preconditions:* result + [0, n) does not overlap with first + [0, n).

- 7 *Effects:* Equivalent to:

```

for (; n > 0; ++result, (void)++first, --n)
    ::new (voidify(*result))
        typename iterator_traits<NoThrowForwardIterator>::value_type(deref-
            move(first));
return {first, result};

```

```

namespace ranges {
    template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-
        for<0> S>
        requires constructible_from<iter_value_t<0>,
            iter_rvalue_reference_t<I>>
        constexpr uninitialized_move_n_result<I, 0>
            uninitialized_move_n(I ifirst, iter_difference_t<I> n, O ofirst, S
                olast);
}

```

- 8 *Preconditions:* [ofirst, olast) does not overlap with ifirst + [0, n).

- 9 *Effects:* Equivalent to:

```

auto t = uninitialized_move(counted_iterator(std::move(ifirst), n),
    default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};

```

```

template<execution-policy Ep, random_access_iterator I,
    nothrow-random-access-iterator O, nothrow-sized-sentinel-for<0> S>
requires constructible_from<iter_value_t<0>, iter_rvalue_reference_t<I>>
uninitialized_move_n_result<I, 0>
    uninitialized_move_n(Ep&& exec, I ifirst, iter_difference_t<I> n,
        O ofirst, S olast);

```

- x *Preconditions:* [ofirst, olast) does not overlap with ifirst + [0, n).

x *Effects:* Equivalent to:

```
auto t = uninitialized_move(std::forward<Ep>(exec),
    counted_iterator(std::move(ifirst), n),
    default_sentinel, ofirst, olast);
return {t.in.base(), t.out};
```

10 [*Note:* If an exception is thrown, some objects in the range `ifirst + [0, n)` are left in a valid but unspecified state. — *end note*]

§ 2.6 Modify `[uninitialized.fill]`

```
template<class NoThrowForwardIterator, class T>
constexpr void uninitialized_fill(NoThrowForwardIterator first,
    NoThrowForwardIterator last, const T&
    x);
+ template<class ExecutionPolicy, class NoThrowForwardIterator, class T>
+ void uninitialized_fill(ExecutionPolicy&& exec,
+     NoThrowForwardIterator first,
+     NoThrowForwardIterator last,
+     const T& x);
```

1 *Effects:* Equivalent to:

```
for (; first != last; ++first)
    ::new (voidify(*first))
        typename iterator_traits<NoThrowForwardIterator>::value_type(x);
```

```
namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S, class T>
        requires constructible_from<iter_value_t<I>, const T&>
        constexpr I uninitialized_fill(I first, S last, const T& x);
    template<nothrow-forward-range R, class T>
        requires constructible_from<range_value_t<R>, const T&>
        constexpr borrowed_iterator_t<R> uninitialized_fill(R&& r, const T&
            x);

+     template<execution-policy Ep, nothrow-random-access-iterator I,
+         nothrow-sized-sentinel-for<I> S, class T>
+     requires constructible_from<iter_value_t<I>, const T&>
+     I uninitialized_fill(Ep&& exec, I first, S last,
+         const T& x);
+     template<execution-policy Ep, nothrow-sized-random-access-range R,
+         class T>
+     requires constructible_from<range_value_t<R>, const T&>
+     borrowed_iterator_t<R> uninitialized_fill(Ep&& exec, R&& r,
+         const T& x);
+ }
}
```

2 *Effects:* Equivalent to:

```
for (; first != last; ++first)
    ::new (voidify(*first)) remove_reference_t<iter_reference_t<I>>(x);
return first;
```

```
template<class NoThrowForwardIterator, class Size, class T>
constexpr NoThrowForwardIterator
uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T&
x);
+ template<class ExecutionPolicy, class NoThrowForwardIterator, class
+ Size, class T>
+ NoThrowForwardIterator
+ uninitialized_fill_n(ExecutionPolicy&& exec,
+ NoThrowForwardIterator first,
+ Size n, const T& x);
```

3 *Effects:* Equivalent to:

```
for (; n--; ++first)
::new (voidify(*first))
typename iterator_traits<NoThrowForwardIterator>::value_type(x);
return first;
```

```
namespace ranges {
template<nothrow-forward-iterator I, class T>
requires constructible_from<iter_value_t<I>, const T>
constexpr I uninitialized_fill_n(I first, iter_difference_t<I> n, con-
st T& x);
}
```

4 *Effects:* Equivalent to:

```
return uninitialized_fill(counted_iterator(first, n), default_sentinel,
x).base();
```

```
template<execution-policy Ep, nothrow-random-access-iterator I, class T>
requires constructible_from<iter_value_t<I>, const T>
I uninitialized_fill_n(Ep&& exec, I first,
iter_difference_t<I> n, const T& x);
```

x *Effects:* Equivalent to:

```
return uninitialized_fill(std::forward<Ep>(exec),
counted_iterator(first, n), default_sentinel, x).base();
```

§ 2.7 Modify [specialized.destroy]

```
template<class T>
constexpr void destroy_at(T* location);
namespace ranges {
template<destructible T>
constexpr void destroy_at(T* location) noexcept;
}
```

1 *Effects:*

- (1.1) — If T is an array type, equivalent to `destroy(begin(*location), end(*location))`.
- (1.2) — Otherwise, equivalent to `location->~T()`.

```
template<class NoThrowForwardIterator>
    constexpr void destroy(NoThrowForwardIterator first,
        NoThrowForwardIterator last);
+ template<class ExecutionPolicy, class NoThrowForwardIterator>
+     void destroy(ExecutionPolicy&& exec,
+         NoThrowForwardIterator first,
+         NoThrowForwardIterator last);
```

2 *Effects:* Equivalent to:

```
for (; first != last; ++first)
    destroy_at(addressof(*first));
```

```
namespace ranges {
    template<nothrow-input-iterator I, nothrow-sentinel-for<I> S>
        requires destructible<iter_value_t<I>>
        constexpr I destroy(I first, S last) noexcept;
    template<nothrow-input-range R>
        requires destructible<range_value_t<R>>
        constexpr borrowed_iterator_t<R> destroy(R&& r) noexcept;

+   template<execution-policy Ep, nothrow-random-access-iterator I,
+       nothrow-sized-sentinel-for<I> S>
+       requires destructible<iter_value_t<I>>
+       I destroy(Ep&& exec, I first, S last) noexcept;
+   template<execution-policy Ep, nothrow-sized-random-access-range R>
+       requires destructible<range_value_t<R>>
+       borrowed_iterator_t<R> destroy(Ep&& exec, R&& r) noexcept;
}
```

3 *Effects:* Equivalent to:

```
for (; first != last; ++first)
    destroy_at(addressof(*first));
return first;
```

```
template<class NoThrowForwardIterator, class Size>
    constexpr NoThrowForwardIterator destroy_n(NoThrowForwardIterator first,
        Size n);
+ template<class ExecutionPolicy, class NoThrowForwardIterator, class
+     Size>
+     NoThrowForwardIterator destroy_n(ExecutionPolicy&& exec,
+         NoThrowForwardIterator first, Size
+         n);
```

4 *Effects:* Equivalent to:

```
for (; n > 0; (void)++first, --n)
    destroy_at(addressof(*first));
return first;
```

```
namespace ranges {
    template<nothrow-input-iterator I>
        requires destructible<iter_value_t<I>>
        constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept;
}
```

5 *Effects*: Equivalent to:

```
return destroy(counted_iterator(std::move(first), n),  
default_sentinel).base());
```

```
template<execution-policy Ep, nothrow-random-access-iterator I>  
requires destructible<iter_value_t<I>>  
I destroy_n(Ep&& exec, I first,  
            iter_difference_t<I> n) noexcept;
```

6 *Effects*: Equivalent to:

```
return destroy(std::forward<Ep>(exec),  
counted_iterator(std::move(first), n), default_sentinel).base());
```

§ 3 References

[P3179R9] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2025-05-29.
C++ parallel range algorithms.

<https://wg21.link/p3179r9>